Random Lunch Recommender

Student: Raimova Alina
Team: Individual
Email: aaraimova@edu.hse.ru
Date: 2026-09-14 **Assignment:** Random Lunch Recommender (Week 1, A01)

Abstract

This homework investigates why a random lunch recommender sometimes displayed a food name without its corresponding icon and whether its 12-item selection sequence behaves uniformly and independently over time. The solution separates HTML, CSS, and JavaScript, replaces three Font Awesome classes unavailable in the pinned Free 6.4.0 stylesheet, and records the registered click handler's complete index sequence. The original 1,000-observation run rejected marginal uniformity (chi-square = 26.264, p = 0.00593), whereas the new pre-specified 100,000-observation sequence did not reject uniformity (p = 0.676), consecutive independence (permutation p = 0.474), or block homogeneity (p = 0.163). Thus, this sequence is consistent with uniform, independent, and stable random selection, but the tests do not prove randomness, and an earlier separate 100,000-observation native run means the evidence across runs remains mixed.

Index Terms - random recommender, JavaScript, chi-square goodness-of-fit.

1. Introduction

Problem statement

The application recommends one of 12 predefined lunch items after a button click. Its visible failure was intermittent: the food name changed correctly, but the icon was blank for some selections. A separate question was whether `Math.floor(Math.random() * lunchMenu.length)` gives every array index the same probability.

Motivation

An intermittent presentation failure can be misdiagnosed as a recommender or randomness failure because it appears only for a subset of outputs. Separating asset compatibility from selection logic makes the defect reproducible and prevents a cosmetic dependency error from being mistaken for an algorithmic bias.

Concrete example

When the original code selected **Ramen**, it inserted `<i class="fas fa-bowl-hot"></i>`. The page loaded Font Awesome Free 6.4.0, but that stylesheet supplied no icon for the class, so the text rendered while the icon remained blank. The same symptom occurred for `fa-pasta` and `fa-bowl`.

Contributions

- Identify and fix a dependency-version mismatch affecting 3 of 12 menu entries, then separate the implementation into `index.html`, `style.css`, and `script.js`.
- Validate the actual registered click handler, preserve both unfavorable and favorable statistical results, and distinguish a correct mapping formula from finite-sample evidence about the pseudo-random source.

2. Related Work

ECMAScript defines `Math.random()` as returning a value greater than or equal to 0 and strictly less than 1, chosen randomly or pseudo-randomly with an approximately uniform distribution using an implementation-defined strategy [1]. Therefore, equal-width interval mapping can make the 12 array indices equiprobable under that contract, although the function is not a cryptographic random-number generator.

For empirical validation, NIST describes the chi-square goodness-of-fit statistic as a comparison of observed and expected bin counts and notes that the approximation requires a sufficient sample size [2]. SciPy implements Pearson's chi-square test for multinomial counts, uses equal expected frequencies by default when none are supplied, and uses `k - 1` degrees of freedom when no parameters are estimated [3]. Font Awesome maintains separate Free and Pro packages, so an icon class must be checked against the exact edition and version used by the page rather than inferred from a plausible name [4].

Alternatives considered

- **Local emoji or SVG assets:** avoid third-party icon-class compatibility, but require local asset design, licensing, and accessibility work. The current fix retains Font Awesome because only three verified substitutions were needed.
- **`crypto.getRandomValues`:** provides stronger random bytes, but is unnecessary for a non-security lunch suggestion and would add conversion logic. `Math.random()` is retained with an explicit non-cryptographic limitation.
- **Exact multinomial or likelihood-ratio testing:** could supplement the analysis, but Pearson's chi-square test is transparent here because all expected counts are far above five and the hypotheses were fixed before the confirmatory run.

3. Method

Approach

The debugging process separated two mechanisms. First, every food name was checked together with the icon class inserted into the DOM and the exact Font Awesome stylesheet loaded by the page. Second, the selection expression and its registered click handler were tested independently of animation timing.

Pipeline

1. Inspect the starter HTML and reproduce the missing-icon symptom for all 12 menu entries.
2. Compare the three failing class names with Font Awesome Free 6.4.0 and replace them with available Free classes.
3. Move structure, styling, and random-selection behavior into separate HTML, CSS, and JavaScript files.
4. Load the real `script.js` in a Node VM, capture its registered click handler, and record the selected food names.
5. Test observed counts against equal probabilities of `1/12` using Pearson's chi-square goodness-of-fit test.
6. Validate the harness by giving the direct formula path and handler path the same seeded pseudo-random sequence and requiring identical count vectors.
7. Preserve a new 100,000-index native sequence, construct its `12 x 12` transition matrix, test adjacent independence analytically and by permutation, and test homogeneity across 100 chronological blocks.

Tools and libraries

- **Font Awesome Free 6.4.0:** icon font loaded by the page; compatibility is constrained to the Free edition [4].
- **Node.js 23.7.0:** executes the actual application script and handler harness.
- **Python 3.7.2 with SciPy 1.7.3:** independently recomputed the saved chi-square results in the current shell using `scipy.stats.chisquare` [3].
- **HTML5, CSS, and browser JavaScript:** implement the user interface without a build framework.

The saved `confirmatory-chi-square-result.json` records SciPy 1.17.1, whereas the current shell reports SciPy 1.7.3. Both produced the same reported statistic and p-value for the preserved counts, but the environment-version mismatch must remain disclosed.

Key design decisions

- **Pin asset validation to the loaded edition:** `fa-bowl-hot`, `fa-pasta`, and `fa-bowl` were replaced by `fa-bowl-food`, `fa-plate-wheat`, and `fa-mug-hot`. This favors verified availability over exact semantic specificity.
- **Test the registered handler:** `distribution-test.js` evaluates the real `script.js` and invokes the captured click listener instead of copying the menu-selection code into a separate experiment.
- **Preserve inconvenient results:** the 100,000-selection protocol specified one run and no replacement based on its p-value, preventing rerun-until-pass behavior.

Configuration

The dataset is the fixed 12-item `lunchMenu` array in `script.js`. The marginal null hypothesis is `P(item_i) = 1/12` for every item, the significance level is `alpha = 0.05`, and the degrees of freedom are `12 - 1 = 11`. Native `Math.random()` runs are unseeded because its public API does not expose seeding; the deterministic harness validation uses seed `20260914`, and the 2,000-replicate permutation analysis uses seed `20260915`. The sequence is divided into 100 consecutive blocks of 1,000, and results are also compared with a Bonferroni threshold of `0.05 / 3 = 0.016667` for the three primary hypotheses. The experiments ran locally on macOS; processor and memory details were not recorded and are not needed for this small computation.

The selection formula is:

```
const randomIndex = Math.floor(Math.random() * lunchMenu.length);
const selectedLunch = lunchMenu[randomIndex];
```

For `n = 12`, a source value `x` in `[0, 1)` maps to index `i` exactly when `x` lies in `[i/12, (i+1)/12)`. These 12 intervals have equal width `1/12`; index 0 includes the lower boundary 0, and index 12 is impossible because 1 is excluded [1].

4. Experiments

Setup

`distribution-test.js` replaces the browser DOM with minimal test objects, loads the application script, captures the button's registered click handler, and invokes it `N` times. Its timeout callback executes immediately because the random draw occurs before the display delay; `harness-validation.js` separately verifies that this simplification does not change the index-to-item mapping. The exploratory run used 1,000 handler invocations. The improvement changed one variable, sample size, to 100,000 while retaining 12 categories, the same selection rule, the same statistic, and `alpha = 0.05`.

For the sequence experiment, `SEQUENCE_EXPERIMENT_PROTOCOL.md` was saved before sampling. `sequence-experiment.js` then recorded all 100,000 emitted indices in order, excluding the automatic page-load draw. `sequence-analysis.py` used the same sequence for marginal uniformity, 99,999 adjacent transitions, a 2,000-replicate permutation test, and a `100 x 12` block-count table.

Results

Table 1. Earlier saved and diagnostic uniformity checks

Run	N	Expected per item	Chi-square	p-value	Decision at 0.05
Exploratory registered-handler run	1,000	83.33	26.264	0.005928	Reject H0
Pre-specified registered-handler run	100,000	8,333.33	31.477	0.000925	Reject H0
Seeded harness/direct-path diagnostic	100,000	8,333.33	4.484	0.9536	Do not reject H0
Direct native diagnostic	1,000,000	83,333.33	11.796	0.3792	Do not reject H0

The 1,000-observation native registered-handler experiment rejects the uniform null hypothesis (`p = 0.005928`). The earlier 100,000-observation native registered-handler experiment also rejects it (`p = 0.000925`). A separate 100,000-observation seeded harness diagnostic does not reject uniformity (`p = 0.9536`), but it uses the injected seeded generator and primarily validates the handler-to-index mapping. These outcomes remain reported rather than being replaced by the new run.

Table 2. Pre-specified 100,000-index sequence analysis

Property and test	Statistic	df	p-value	Decision at 0.05
Marginal uniformity, chi-square goodness-of-fit	8.410	11	0.6762	Fail to reject H0
Consecutive independence, chi-square	121.676	121	0.4657	Fail to reject H0
Consecutive independence, 2,000 permutations	947 extreme	-	0.4738	Fail to reject H0
Homogeneity across 100 blocks	1134.894	1089	0.1625	Fail to reject H0

All decisions are unchanged at the Bonferroni threshold `0.016667`. The minimum expected transition frequency was `677.01`, and the minimum expected block/category frequency was `82.28`, so the analytical chi-square approximations were not threatened by small expected cells.

Comparison with baseline

The original 1,000-observation native run rejected uniformity, while the new pre-specified 100,000-index native sequence did not reject marginal uniformity and showed no detected serial dependence or time drift. This is the requested baseline-to-larger-sample comparison, but sample size alone does not guarantee a particular p-value: the earlier, separately preserved 100,000 native run rejected uniformity. The differing native runs cannot be merged selectively or used to claim proof. Together with the seeded equivalence test, the new transition and homogeneity results weaken the hypothesis of an index-mapping or obvious sequence-order defect, while the cross-run discrepancy remains unexplained.

Verification

The deterministic validation gave the direct path and registered-handler path the same seed (`20260914`) and confirmed exact equality of all 12 counts across 100,000 selections. This verifies that the harness preserves the mapping performed by the application. For the new sequence, independent assertions confirmed 100,000 valid indices, a count sum of 100,000, a transition-matrix sum of 99,999, 100 block totals of exactly 1,000, transition margins consistent with the first and last index, and a manual marginal statistic of `8.410160`. Re-running the analysis with the same saved sequence and permutation seed produced a byte-for-byte identical JSON result.

A transparent manual check is prepared for student verification using the largest confirmatory contribution, Burger:

```
Observed O = 8666
Expected E = 100000 / 12 = 8333.3333
(O - E)^2 / E = (8666 - 8333.3333)^2 / 8333.3333 = 13.280053
```

5. Discussion

Failure case

Failure: for some objects in the array (Ramen, Pasta and Soup) there is no icon in the Font Awesome service to provide pictures for this names **Change:** replace the unavailable classes and separate the page into structure, design, and behavior files.
Confirmation: all 12 corrected icon classes rendered in the same browser check, and the random-selection handler continued to return valid menu items.

Prompt correction

The original product request did not specify the Font Awesome edition or version and did not require exhaustive asset validation. A safer prompt is:

> Build a random lunch generator using separate `index.html`, `style.css`, and `script.js` files. Use only icons available in Font Awesome Free 6.4.0 from the provided CDN. Before returning the code, verify every icon class against that exact stylesheet and render-test all 12 menu items in a browser. If an exact food icon is unavailable, use a verified generic fallback and list the substitution. Do not use Pro-only, version-mismatched, or guessed icon names. Add a test that fails when any menu item has no rendered icon glyph.

The improvement is measurable: the original implementation had 3 failing icon entries out of 12, while the corrected implementation had 0 out of 12 in the browser audit.

What worked

- Separating the visual dependency failure from the random index calculation produced a specific, reproducible root cause.
- The seeded handler/direct-path comparison established that the test harness does not reorder or miscount menu items.

Statistical conclusion

For the pre-specified sequence of 100,000 selections, the statistical tests found no significant departure from marginal uniformity, independence of consecutive selections, or homogeneity across time blocks. Therefore, this sequence is consistent with the tested random model. In statistical terms, the three null hypotheses were **not rejected**; they were not proved true. The conclusion is limited to the observed sequence and the tested properties, and it does not establish that `Math.random()` is random in every run or under every possible randomness test. The earlier rejecting runs are retained because their disagreement with the latest experiment is part of the evidence rather than a result that can be discarded.

Next improvement

Pre-register at least 100 independent batches, preserve every batch, and compare the fraction of p-values below 0.05 with the nominal false-positive rate. Repeat the same browser-level protocol in at least two engines and record their versions. This tests whether the rejection pattern persists without rerunning until a preferred p-value appears.

6. AI Usage Disclosure

AI tools used

- **OpenAI Codex:** GPT-5.6 Sol (High).

How AI was used

- Inspected the starter and student repositories.
- Diagnosed Font Awesome class incompatibility, proposed verified replacements, and separated HTML, CSS, and JavaScript.
- Created the handler simulation, deterministic harness check, recorded-sequence experiment, transition and block analyses, chi-square and permutation tests, chart, and this report draft.

What I personally verified

1. Clicking at the menu and found problem
2. Read the statistical examples to make write plan for agent
3. Cross-checked the fixes of agent for random menu

References

[1] Ecma International, "ECMAScript 2025 Language Specification," sec. 21.3.2.28, *Math.random*, 2025. [Online]. Available: https://tc39.es/ecma262/2025/multipage/numbers-and-dates.html#sec-math.random. [Accessed: Sep. 15, 2026].

[2] NIST/SEMATECH, "Chi-Square Goodness-of-Fit Test," *e-Handbook of Statistical Methods*, sec. 1.3.5.15. [Online]. Available: https://www.itl.nist.gov/div898/handbook/eda/section3/eda35f.htm. [Accessed: Sep. 15, 2026].

[3] SciPy Developers, "scipy.stats.chisquare," *SciPy v1.17.0 Manual*. [Online]. Available: https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.stats.chisquare.html. [Accessed: Sep. 15, 2026].

[4] Fonticons, Inc., "Using a Package Manager," *Font Awesome Docs Version 6*. [Online]. Available: https://docs-v6.fontawesome.com/web/setup/packages. [Accessed: Sep. 15, 2026].